

Biological Intuition on Digital Hardware: An RTL Implementation of Poisson-Encoded SNNs for Static Image Classification

Yogeeth GK

Department of Electronics and Communication Engineering,
National Institute of Technology, Durgapur

ABSTRACT

This report presents a comprehensive hardware-software co-design investigation into Spiking Neural Networks (SNNs) for the MNIST handwritten digit classification task. We motivate SNNs by establishing the fundamental energy bottlenecks of conventional Artificial Neural Networks (ANNs), followed by the mathematical formulation of the Leaky Integrate-and-Fire (LIF) neuron and surrogate gradient training. Two encoding strategies—Poisson rate coding and Time-to-First-Spike (TTFS)—are evaluated in software. Crucially, this report transitions from theoretical software models to physical silicon by detailing the RTL implementation of a custom SNN inference accelerator on a Xilinx Zynq-7000 SoC using the PYNQ framework. We explore the critical architectural challenges of deploying SNNs on FPGAs, including LFSR-based stochastic MAC generation, AXI-Stream DMA interfacing, fixed-point weight quantization, and state-machine design for inter-image membrane potential resets. The physical hardware deployment ultimately achieves an 84.80% classification accuracy, closely tracking our 90.69% deterministic software baseline. By analyzing the discrepancies between software simulations and stochastic hardware execution, this work provides a practical blueprint for deploying highly efficient, bare-metal neuromorphic accelerators.

Keywords: Spiking Neural Networks, Hardware-Software Co-Design, LIF Neuron, FPGA Accelerator, Verilog RTL, Poisson Encoding, AXI-Stream, Neuromorphic Computing.

1. INTRODUCTION

1.1 The Energy Crisis in Modern AI

The dominant paradigm in artificial intelligence today is the deep Artificial Neural Network — a mathematical architecture inspired loosely by the brain but implemented as layered matrix multiplications on silicon. The results have been extraordinary. Convolutional neural networks match human-level accuracy on image recognition. Transformer-based language models generate fluent text, write code, and reason across domains. But there is an uncomfortable number that sits behind these achievements.

The human brain, performing real-time perception, reasoning, language, and motor control simultaneously, consumes approximately **20 watts** of power. A single NVIDIA A100 GPU — the workhorse of modern deep learning — draws **400 watts**. Training GPT-3 consumed an **estimated energy equivalent to 500 homes**

running for a year. Running inference on large language models at production scale requires dedicated power infrastructure.

This gap — four to five orders of magnitude in energy efficiency between biological and artificial intelligence — is not a transient engineering limitation. It reflects a fundamental architectural difference in how computation is organised.

Neuromorphic computing

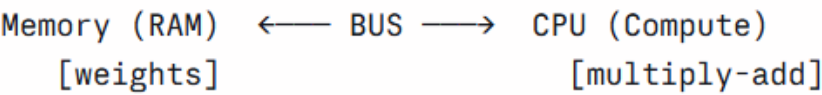
Neuromorphic computing (also called neuromorphic engineering) is an approach to computing that mimics the structure and function of the human brain. It entails designing hardware and software that simulate neural and synaptic structures to process information in a brain-like manner

Biological Element	Neuromorphic Equivalent	Function
Neuron	Artificial spiking neuron	Stores/accumulates charge; fires ("spikes") when threshold is reached
Synapse	Transistor-based synaptic device / memristor	Connects neurons; has programmable weight & delay values
Spike timing	Event-driven signaling	Information propagates asynchronously based on spike events
Neuroplasticity	Spike-timing-dependent plasticity (STDP)	Synaptic weights adapt based on activity patterns

1.2 The Von Neumann Bottleneck

Every modern computer — whether a CPU, GPU, or TPU — is built on the Von Neumann architecture: a strict separation between the processing unit (CPU) and memory (RAM), connected by a communication bus.

This separation creates the **Von Neumann bottleneck**:



Data must physically travel across the bus on every operation. Even if the CPU finishes instantly, it waits for data. The energy cost of this travel is severe:

Operation	Energy Cost
32-bit ADD (compute)	1 pJ
Load from SRAM (cache)	5 pJ
Load from DRAM (RAM)	100–1000 pJ

Note: The Above data is not part of research

In modern deep learning workloads, approximately **90% of energy is consumed moving data**, not computing on it. The mathematical operations — the multiply-accumulate (MAC) instructions that actually implement

neural network inference — consume only the remaining 10%.

For neural networks specifically, this bottleneck is structural. Every forward pass through a network requires fetching the full weight matrix from memory for every layer, multiplying it by every input, and storing the result — on every single inference, for every single input.

Aspect	Von Neumann	Neuromorphic
Processing Model	Sequential instruction execution	Parallel, event-driven spiking
Memory/Compute	Separated (CPU ↔ RAM bottleneck)	Integrated in neurons/synapses
Power Usage	Continuous power draw	Power only on spike events
Data Flow	Linear, deterministic	Asynchronous, stochastic
Fault Tolerance	Single-point failures critical	Degraded gracefully (distributed)
Learning	Requires explicit retraining	Online, adaptive plasticity

1.3 Why Standard ANNs Are Inefficient

A conventional ANN layer computes:

$$\mathbf{a}^{(l)} = f\left(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\right)$$

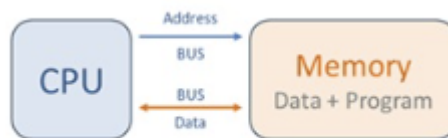
Where $f(\cdot)$ is a nonlinear activation (ReLU, sigmoid, etc.), $\mathbf{W}$ are the learned weights, and $\mathbf{a}$ are the activations.

The key property: Every neuron produces a continuous real-valued output at every forward pass. A neuron in a 512-neuron hidden layer always outputs a float, whether its contribution is large (0.95) or essentially zero (0.0001). There is no sparsity. There is no silence. The hardware does the same amount of work regardless of whether the input is informative or noise.

For an MNIST classification network with architecture $784 \rightarrow 256 \rightarrow 10$:

$$\text{MACs per inference} = 784 \times 256 + 256 \times 10 = 203,264$$

All 203,264 multiply-accumulate operations execute on every single image. Every image costs the same, regardless of complexity.



Von Neumann Architecture

1.4 The Biological Solution — Event-Driven Computation

The brain solved this problem 500 million years ago.

Biological neurons do not produce continuous floating-point outputs. They produce **spikes** — brief, all-or-nothing electrical pulses lasting approximately 2 milliseconds. Between spikes, a neuron is electrically

silent. It consumes almost no dynamic power. No computation propagates downstream. The hardware is idle. Information in the brain is encoded not in the amplitude of a signal (spikes are all the same amplitude) but in **when** neurons fire and **which** neurons fire. This is the fundamental shift: information is temporal and sparse, not spatial and dense.

The consequence: at any given moment, only 1–5% of neurons in a cortical area are active. The remaining 95–99% are silent, consuming no dynamic power. Computation is proportional to the information content of the input, not the size of the network.

This is the design philosophy that Spiking Neural Networks (SNNs) aim to capture in artificial hardware.

Biological Mechanics of an Action Potential ?

Continuous vs. Discrete Event Processing

Every thought you're having is the result of **neurons firing electrical pulses**. These pulses are called **action potentials** or **spikes**. A spike is binary — it either happens or it doesn't. The information is NOT in the amplitude of the spike (all spikes look roughly the same), it's in:

- **When** the spike happens (timing)
- **How often** spikes happen (frequency/rate)
- **Which neurons** spike (identity)
- **The pattern** across populations of neurons

A neuron receives inputs from thousands of other neurons through **synapses**. These inputs slowly charge up its **membrane potential** (like charging a capacitor). When that voltage hits a **threshold**, the neuron fires a spike — and then the membrane potential resets. This whole mechanism is called **integrate-and-fire**.

1.5 Scope of This Report

This report addresses the following questions through theory, mathematical derivation, and experiment:

- What is the mathematical model of a spiking neuron, and how does it capture biological dynamics?
- How do we convert static image data (MNIST) into spike trains? What encoding strategies exist and what are their tradeoffs?
- How do we train an SNN using backpropagation when the spike function is non-differentiable?
- What do the experimental results tell us about the relationship between timesteps, encoding, and accuracy?
- What are the implications for hardware deployment, specifically FPGA and neuromorphic chips?

2. BACKGROUND — FROM ANN TO SNN

2.1 The Three Generations of Neural Networks

Wolfgang Maass formally articulated the generational framework of neural networks in his 1997 paper "Networks of Spiking Neurons: The Third Generation of Neural Network Models":

- **First Generation (1940s–1960s):** McCulloch-Pitts threshold neurons. Binary inputs, binary outputs. Could compute Boolean logic. No continuous learning, no real-valued representation.
- **Second Generation (1980s–present):** Neurons with continuous activation functions (sigmoid, tanh, ReLU). Real-valued outputs. Learnable via backpropagation. This is what modern "deep learning" refers to.
- **Third Generation (1997–present):** Spiking neurons. Binary spike outputs. Temporal dynamics.

Information carried in spike timing and identity, not amplitude. Learnable with surrogate gradients (2018–present).

The computational distinction is precise. Maass proved that spiking neurons with temporal coding can compute functions that rate-coded networks cannot represent efficiently — because time is an additional dimension of information unavailable to second-generation networks.

2.2 The Fundamental Contrast

Property	ANN (2nd Gen)	SNN (3rd Gen)
Neuron output	Continuous float	Binary spike (0 or 1)
Output at each step	Always active	Only on threshold crossing
Time	Not modeled	Fundamental variable
Information carrier	Activation value	Spike timing + identity
Hardware computation	Dense MACs	Sparse accumulations
Energy per neuron	Constant	Proportional to firing rate
Biological plausibility	Loose	High

SNNs have become the center of attention because:

- **Neuromorphic hardware is finally here** — Intel Loihi 2, IBM TrueNorth, BrainScaleS are real chips
- **Surrogate gradient methods** cracked the training problem (2018–2022)
- **DVS event cameras** produce data that is natively SNN-compatible
- **Energy efficiency** is now a first-class research concern, not an afterthought
- **Transformer-based SNNs** (Spikformer, etc.) are now approaching ANN accuracy
- **Edge AI** needs models that run on microwatts — SNNs are the best candidate

3. THE MATHEMATICAL FOUNDATION — LIF NEURON MODEL

3.1 From Biology to Circuit - [Prototype](#)

A biological neuron's membrane behaves like a capacitor in parallel with a resistor — the standard RC circuit model. Synaptic inputs deliver charge to this membrane. The membrane voltage rises as charge accumulates and decays as it leaks through the resistance. When the voltage reaches a threshold, an action potential (spike) is triggered and the membrane resets.

This RC dynamics is described by:

$$C_m \frac{dV(t)}{dt} = -\frac{V(t) - V_{\text{rest}}}{R_m} + I(t)$$

Multiplying both sides by R_m and defining $\tau_m = R_m * C_m$ (the membrane time constant):

$$\tau_m \frac{dV(t)}{dt} = -(V(t) - V_{\text{rest}}) + R_m I(t)$$

Setting $V_{\text{rest}} = 0$ for simplicity, we get the Leaky Integrate-and-Fire (LIF) differential equation:

$$\tau_m \frac{dV(t)}{dt} = -V(t) + R_m I(t)$$

This is the **Leaky Integrate-and-Fire (LIF)** differential equation. The term $-V(t)$ is the "leak" — the natural tendency of the membrane to return to rest in the absence of input.

Fire-and-reset rule:

$$S(t) = \Theta(V(t) - \theta) = \begin{cases} 1 & \text{if } V(t) \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

Where $\Theta(\cdot)$ is the Heaviside step function and θ is the firing threshold. Upon spiking: $V(t) \leftarrow V_{\text{reset}}$ (commonly 0).

3.2 Discretisation — The Implementable Form

For digital hardware (GPU, FPGA, neuromorphic chip), we discretise time into steps of size Δt . Using the forward Euler method on the LIF ODE:

$$V[t] = V[t - 1] + \frac{\Delta t}{\tau_m} (-V[t - 1] + R_m I[t])$$

$$V[t] = \left(1 - \frac{\Delta t}{\tau_m}\right) V[t - 1] + \frac{\Delta t}{\tau_m} R_m I[t]$$

Define the **decay factor** (also called leak or beta):

$$\beta = 1 - \frac{\Delta t}{\tau_m} = e^{-\Delta t / \tau_m} \in (0, 1)$$

Then the complete discrete LIF model becomes:

$$V[t] = \beta \cdot V[t - 1] + W \cdot X[t]$$

$$S[t] = \Theta(V[t] - \theta)$$

$$V[t] \leftarrow V[t] - S[t] \cdot \theta \quad (\text{soft reset})$$

These three equations are the entire LIF model. Everything else in SNN design is built on top of this foundation.

3.3 Understanding Beta — The Memory Parameter

Beta is the single most important SNN-specific hyperparameter. It physically represents how fast the membrane potential leaks back to rest.

- $\beta = 0.99 \Rightarrow$ retains 99% of charge per step $\Rightarrow$ long memory

- $\beta = 0.90 \Rightarrow$ retains 90% of charge per step $\Rightarrow$ medium memory (sweet spot)
- $\beta = 0.50 \Rightarrow$ retains 50% of charge per step $\Rightarrow$ short memory

The steady-state membrane potential under constant input I_0 is:

$$V_{\text{steady}} = \frac{W \cdot X}{1 - \beta}$$

This means: to ensure neurons can fire (reach threshold θ) under typical inputs, we need:

$$W \cdot X \approx \theta \cdot (1 - \beta)$$

This provides the weight initialisation principle used in our implementation.

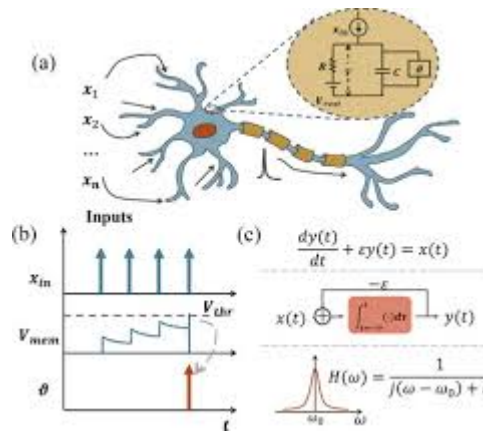
Research finding: Cross-sweeping β and θ together achieves a 48% reduction in hardware inference latency with only 2.88% accuracy trade-off — because higher θ and lower β together reduce spike count, directly reducing synaptic operations on chip.

3.4 Threshold — The Selectivity Gate

The firing threshold θ controls how selective each neuron is:

- **Low θ :** Neuron fires frequently $\rightarrow$ rich information flow, but high spike count $\rightarrow$ low energy efficiency
- **High θ :** Neuron fires rarely $\rightarrow$ sparse, efficient, but may discard useful information

Experimental observation from our results: At $T=20$ with threshold=255 and rate coding, accuracy reaches 90.13%. At $T=50$ with the same threshold, accuracy drops to 88.42%. This counterintuitive result (more timesteps = lower accuracy) reveals that $T=50$ with Poisson encoding produces excessive spike noise that overwhelms threshold integration.



4. SPIKE ENCODING — CONVERTING PIXELS TO SPIKES

1. The Importance of Encoding (Bridging the Gap)

Encoding is the mandatory translator between the continuous, static real world and the discrete, dynamic neuromorphic world.

An MNIST pixel is a static scalar value (e.g., 0.85). A standard ANN processes this 0.85 instantly in a single clock cycle by multiplying it against a 32-bit floating-point weight.

An SNN cannot do this. An SNN's neurons communicate strictly using **1-bit binary events (spikes)**. You cannot push the number 0.85 through a 1-bit wire. The encoding function is important because it answers the question: *How do we express the magnitude of a continuous number using only 1s and 0s?*

2. Why Add a Time Dimension? (The Significance)

If you only have a 1-bit wire, the only way to transmit a value larger than 1 is to **use time**. The time dimension is what gives SNNs their unique properties. Its significance breaks down into three key engineering advantages:

Information Density in Binary Systems:

Since the amplitude of a spike is always exactly 1, the "information" must be hidden in the temporal arrangement of the spikes. If a pixel is very bright (high intensity), the encoder might fire many spikes rapidly over a set time window (**Rate Coding / Poisson**), or it might fire exactly one spike very early in the time window (**Temporal Coding / TTFS**). Time becomes the carrier of the payload.

Energy Efficiency via Sparsity (Hardware Impact):

By adding time, SNNs achieve *temporal sparsity*. In your FPGA, instead of forcing the MAC unit to perform heavy 8-bit $\times$ 16-bit multiplications every cycle for every pixel, the encoder turns pixels into sparse 1s and 0s over time. When the input is 0 (no spike), the hardware does absolutely nothing—it skips the operation, saving power. When the input is 1 (spike), the hardware only has to perform a simple addition (Accumulator += Weight), completely eliminating power-hungry digital multipliers.

Accumulation and Leak Dynamics:

Real neurons don't process data instantly; they accumulate electrical charge over time and "leak" charge if stimulation stops. Adding a time dimension allows the Leaky Integrate-and-Fire (LIF) model to actually behave like a physical capacitor. Over Δt timesteps, the neuron slowly integrates the incoming spikes. This temporal integration provides inherent noise-filtering; random "ghost" spikes will leak away over time, while persistent, meaningful spike trains will trigger an output.

In a spiking system, since every spike is identical in amplitude (it's just a 1 or a 0), you cannot encode information in the *size* of the spike. Instead, the information is presented in the **distance between the spikes**—a metric known in neuromorphic engineering as the **Inter-Spike Interval (ISI)**.

4.1 Rate Coding — Poisson Encoding

Poisson encoding is a type of **rate coding**. This means the *information* is represented by the *number of spikes* (the firing rate) over a specific time window, rather than the exact timing of those spikes.

- A high input value (e.g., a bright white pixel) results in a high firing rate (many spikes).
- A low input value (e.g., a dark gray pixel) results in a low firing rate (few spikes).

Poisson encoding models the generation of spikes as a **Poisson process**. In a homogeneous Poisson process, the generation of a spike at any given moment is completely independent of when the previous spikes occurred.

If you have an input value that translates to a firing rate r (in Hertz, or spikes per second), the probability of a spike occurring in a very small time window Δt is proportional to that rate:

$$P(\text{spike in } \Delta t) \approx r \cdot \Delta t$$

- **Advantage:** It allows for the conversion of complex, real-valued input into a simplified, sparse representation that is directly compatible with SNNs.
- **Limitation:** It is a stochastic (probabilistic) method, meaning the same input can produce different spike trains in different trials, which introduces noise.

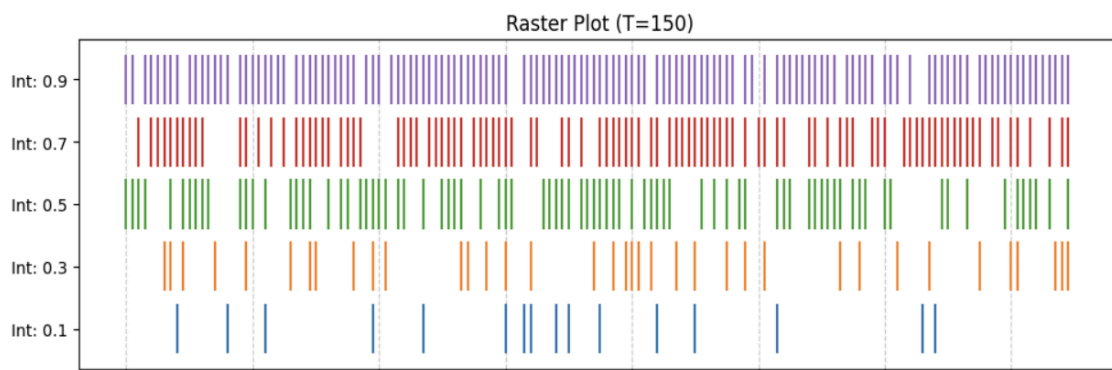
The Poisson Process (Simplified for Hardware)

Given:

- Input value $x \in [0, 1]$ (normalized) ## Normalization Is Optional If Max value is Low enough
- Time discretized into steps Δt (e.g., 1 clock cycle)
- Firing probability $p = x$ per timestep

For each timestep t :

```
Generate random number  $r \in [0, 1]$ 
if ( $r < x$ ) → emit spike at time  $t$ 
else → no spike
```



Clarification: This is technically a *Bernoulli process* (discrete time), but converges to a Poisson process as $\Delta t \rightarrow 0$. In hardware, we use the Bernoulli approximation because it's clock-synchronous and implementable

When you feed a **28x28 image** (like a handwritten digit from the MNIST dataset) into an SNN, you are fundamentally changing the shape of the data. In a standard artificial neural network, the input is a flat 1D array of 784 pixels. In an SNN, we add a **time dimension**.

The 28x28 static image becomes a "video" of binary spikes over T time steps.

Visualizing the Stability

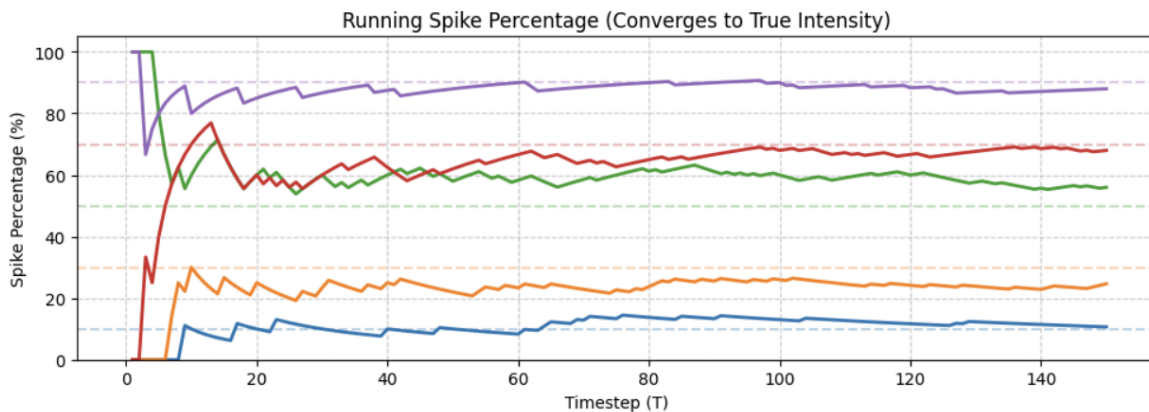
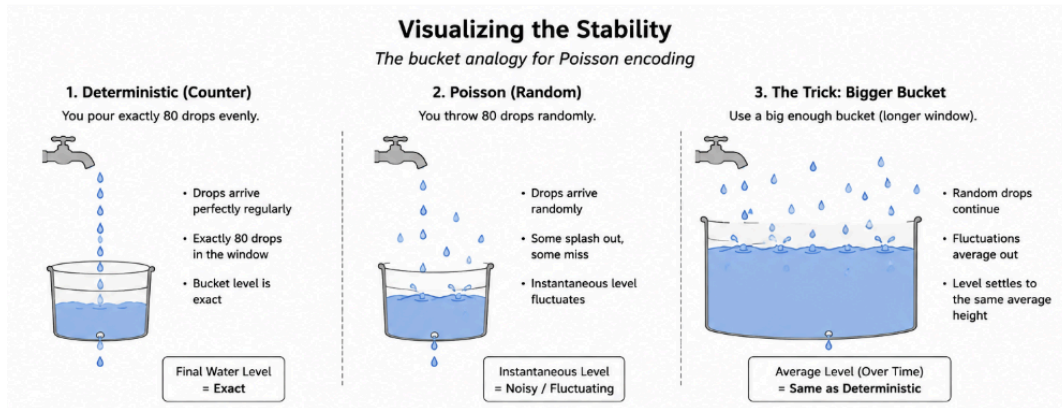
Imagine you are measuring rainfall with a bucket that has a hole in it.

- **Deterministic (Counter):** You pour exactly 80 drops. The bucket level is exact.
- **Poisson (Random):** You throw 80 drops randomly. Some splash out, some miss.
- **The Trick:** If you use a **big enough bucket** (integration window), the water level settles to the same average height.

Real-World Examples of Poisson Processes:

- Raindrops hitting the ground
- Phone calls arriving at a call center
- Radioactive decay events
- Mutations in DNA
- Births in a hospital

Common theme: Events happen **randomly** but with a **predictable average rate**.



Why is Neural Encoding Called "Poisson"?

- When scientists first started sticking electrodes into the brains of animals to measure actual biological neurons, they noticed the neurons didn't fire in perfect, rhythmic pulses. They fired irregularly and randomly—exactly like the clicks on a Geiger counter or raindrops on a sidewalk.
- Biological neurons act as a Poisson process. The "average rate" of the spikes represents the information (like how loud a sound is, or how bright a light is), but the exact timing of each individual electrical spike is random and independent.
- When we use the term **Poisson encoding** in engineering, we are explicitly saying: *"We are going to encode our data by generating spikes randomly so that it perfectly mimics the statistical randomness of biological brains."*
- When you use that random Linear Feedback Shift Register (LFSR) and the comparator we talked about, you are artificially building a raindrop generator. The LFSR provides the random independence, and your input pixel value dictates the average rate of the "rain."

Why are we generating a random number for spike

Why is Randomness Helping Here?

You might ask: *"Why not just make a deterministic counter? If $x=0.8$, fire exactly 8 times every 10 cycles. Why add random noise?"*

This is the most important engineering insight. We use randomness for **three critical reasons**:

Reason 1: Preventing "Resonance" and Sync Issues

Imagine you have 1,000 neurons all receiving input $x=0.8$.

- **Deterministic:** All 1,000 neurons fire at cycle 1, 2, 3, 4, 5, 6, 7, 8... then rest at 9, 10.

- **Result:** Massive current spike on the power grid at cycles 1-8. Voltage drops. System crashes.
- **Random (Poisson):** Neuron A fires at cycles 1, 3, 5... Neuron B fires at 2, 4, 6...
- **Result:** The power draw is **smoothed out** over time. This is crucial for low-power hardware.

Reason 2: Avoiding Harmonic Interference

In digital systems, periodic signals (like "fire every 10 cycles") create **harmonics** (specific frequencies).

- If your SNN has many layers, these periodic spikes can interfere with each other constructively or destructively.
- Randomness spreads the energy across the frequency spectrum (white noise), preventing weird resonance bugs in the network.

Cycle	RNG Output (Random)	Threshold	Comparison (RNG < Thresh?)	Output Spike	Cumulative Count
1	150	204	Yes	1	1
2	210	204	No	0	1
3	050	204	Yes	1	2
4	190	204	Yes	1	3
5	220	204	No	0	3
6	100	204	Yes	1	4
7	010	204	Yes	1	5
8	250	204	No	0	5
9	180	204	Yes	1	6
10	090	204	Yes	1	7

Observation:

- In 10 cycles, we expected 8 spikes (10×0.810×0.8).
- We got 7 spikes.
- **Error:** 1 spike (10% error).
- **If we wait 100 cycles:** The error drops to ~3%.
- **If we wait 1000 cycles:** The error drops to ~1%.

The Downstream Neuron: The next neuron doesn't look at Cycle 1. It has a "membrane potential" capacitor.

- Cycle 1: Capacitor charges +1.
- Cycle 2: Capacitor leaks -0.1.
- Cycle 3: Capacitor charges +1.
- **Result:** The capacitor voltage rises smoothly, averaging out the gaps where no spike occurred.

Taken from Internet The values represented here or not from our research

RNG Type	Pros	Cons	Best For
Single LFSR	Minimal LUTs (~20), high fmax (>300 MHz)	Short period, predictable patterns	Low-accuracy, resource-constrained edge
Dual LFSR + XOR	Better randomness, longer period	+~50% LUTs, slight fmax drop	High-accuracy SNNs, noise-robust applications
TRNG (ring oscillator)	True randomness	High power, area, calibration needed	Security-critical neuromorphic systems

Way of Encoding

- When encoding an image (e.g., MNIST $28 \times 28 = 784$ pixels), how you manage RNGs is critical.
- **Should We Use same rng value for All pixels (Example : $28 * 28$)**

If We use

Pixel A 146 , Pixel B 200

Time Step	Shared RNG	Pixel A Spike	Pixel B Spike
T_1	100	1	1
T_2	255	0	0
T_3	145	1	1
T_4	245	0	0

It Has same Spike pattern It would be harder for neuron to identify

Time Step	RNG	Pixel A Spike	RNG	Pixel B Spike
T_1	100	1	147	1
T_2	255	0	45	1
T_3	145	1	180	1
T_4	245	0	230	0

Same RNG: They will fire at **identical** cycles almost all the time. The neuron sees them as identical.

- **Different RNG:** Their spike trains will be **decorrelated**. Sometimes A fires alone, sometimes B fires alone.
- **Why this matters:** The downstream neuron can detect the **difference** in timing density. It can learn that "When A fires alone, it means edge left. When B fires alone, it means edge right."

Different RNG (XOR Mask) for Each Pixel (The Solution)

You XOR the RNG with the Pixel ID. Now each pixel has a **different effective random number**.

- Generate single random number and Xor with different pixel id Will get different random number for all pixel
- $\text{RNG_pixel} = \text{LFSR_output} \oplus \text{pixel_ID}$

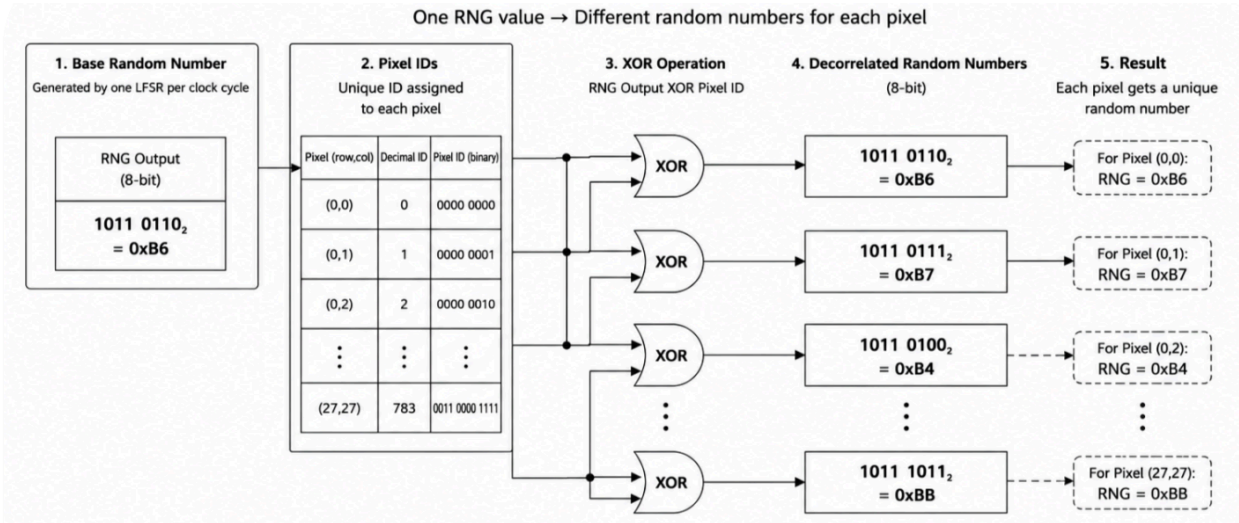
Critical design decision: separate RNG per pixel vs. shared RNG.

Using the same RNG value for all 784 pixels at each timestep creates:

1. **Spatial correlation:** If pixel A and pixel B both have value 0.8, they fire at identical timesteps. Downstream neurons cannot distinguish their contributions.
2. **STDP failure:** Spike-timing-dependent learning requires precise timing differences between pre and post synaptic spikes. Identical timing means $\Delta t = 0$ — the STDP rule is

undefined, and no learning occurs.

3. **Power surges:** All 784 pixels firing simultaneously at the same clock cycle causes catastrophic current spikes on the power rail, causing voltage droop and potential timing violations.



--- Input Current Analysis (Direct Binary, 400 Samples) ---

Digit	Avg Current	Min	Max	Status
0	1.1	-1.9	2.4	
1	0.7	-4.1	1.8	
2	1.0	-5.7	3.1	
3	0.9	-1.6	3.3	
4	1.0	-2.6	2.9	
5	0.7	-3.9	5.3	
6	1.3	-3.0	3.4	
7	1.2	-2.4	3.7	
8	0.4	-1.3	1.5	
9	0.6	-5.5	1.7	

Digit	Avg Current	Min	Max	Status
0	1.1	-1.9	2.4	
1	0.7	-4.1	1.8	
2	1.0	-5.7	3.1	
3	0.9	-1.6	3.3	
4	1.0	-2.6	2.9	
5	0.7	-3.9	5.3	
6	1.3	-3.0	3.4	
7	1.2	-2.4	3.7	
8	0.4	-1.3	1.5	
9	0.6	-5.5	1.7	

Our Input Current Analysis (Direct Binary) directly supports this: the min/max current ranges per digit class (e.g., digit 1 ranges from -4.1 to +1.8, digit 5 from -3.9 to +5.3) confirm that each digit class produces a statistically distinct current distribution. An FPGA must be designed to store values across this full range. For a maximum observed current of ~5.3 and minimum of ~-5.7, a **signed 4-bit fixed-point representation** (range: -8 to +7.875 in steps of 0.0625) is sufficient. If precision requirements tighten, a signed 8-bit representation handles the range comfortably with room for accumulation overflow.

4.2 Temporal Coding — Time-to-First-Spike (TTFS)

Principle: Brighter pixel fires earlier. Each pixel fires at most once.

$$t_{\text{spike}}(p) = \text{round}((1 - p) \times (T - 1))$$

$$S[t, p] = \begin{cases} 1 & \text{if } t = t_{\text{spike}}(p) \text{ and } p > 0 \\ 0 & \text{otherwise} \end{cases}$$

For T=10 (Fixed)

Pixel Value p	Spike Time	Spike Train (T=10)
1.00	0	[1,0,0,0,0,0,0,0,0,0]
0.75	2	[0,0,1,0,0,0,0,0,0,0]
0.50	5	[0,0,0,0,0,1,0,0,0,0]
0.25	7	[0,0,0,0,0,0,0,1,0,0]
0.00	never	[0,0,0,0,0,0,0,0,0,0]

5. THE TRAINING PROBLEM AND SOLUTION

5.1 Why Standard Backpropagation Fails

Neural network training via backpropagation requires computing the gradient of the loss with respect to each weight:

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial S[t]} \cdot \frac{\partial S[t]}{\partial V[t]} \cdot \frac{\partial V[t]}{\partial W}$$

The middle term requires the derivative of the spike function:

$$\frac{\partial S[t]}{\partial V[t]} = \frac{d}{dV} \Theta(V - \theta) = \delta(V - \theta)$$

The Dirac delta is zero almost everywhere and undefined (infinite) at $V=\theta$. The chain rule product collapses:

$$\frac{\partial \mathcal{L}}{\partial W} = (\text{something}) \times 0 \times (\text{something}) = 0$$

No gradient. No weight update. No learning. This is the **dead gradient problem** — the fundamental barrier that blocked deep SNN training for over two decades.

5.2 The Surrogate Gradient Method

The solution, introduced cleanly by Neftci, Mostafa, and Zenke (2019), is conceptually simple:

During the forward pass: Use the real Heaviside step function. Spikes remain binary. Physics is correct.

During the backward pass only: Replace $\delta(V-\theta)$ with a smooth, differentiable approximation $\hat{g}(V)$ that is non-zero near the threshold.

The modified gradient becomes:

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^T \frac{\partial \mathcal{L}}{\partial S[t]} \cdot \underbrace{\hat{g}(V[t] - \theta)}_{\text{surrogate}} \cdot \frac{\partial V[t]}{\partial W}$$

5.3 Common Surrogate Functions

Arctangent (ATan) — used in our experiments:

$$\hat{g}(V) = \frac{\alpha}{1 + (\pi\alpha(V - \theta))^2}$$

Properties:

- Peak height: α at $V=\theta$
- Half-width: $1/\pi\alpha$
- $A = 2.0$ standard starting point

Fast Sigmoid:

$$\hat{g}(V) = \frac{1}{(1 + k|V - \theta|)^2}$$

Research shows the fast sigmoid yields higher sparsity (fewer spikes) with similar accuracy — preferable for hardware deployment.

Triangle (Piecewise Linear):

$$\hat{g}(V) = \max(0, 1 - |V - \theta|)$$

Simplest implementation. No parameters. Good for FPGA where complex functions are expensive.

5.4 Backpropagation Through Time (BPTT) — Full Derivation

For an SNN unrolled over T timesteps, the complete gradient computation:

Step 1 — Forward pass:

$$V[t] = \beta \cdot V[t-1] \cdot (1 - S[t-1]) + W \cdot x[t] + b$$

$$S[t] = \Theta(V[t] - \theta)$$

Step 2 — Loss:

$$\mathcal{L} = \text{CrossEntropy} \left(\sum_{t=1}^T S_{\text{out}}[t], y_{\text{true}} \right)$$

Step 3 — Backward pass (using surrogate):

Gradient of loss w.r.t. spike at time t :

$$\delta_S[t] = \frac{\partial \mathcal{L}}{\partial S[t]}$$

Gradient through the surrogate:

$$\delta_V[t] = \delta_S[t] \cdot \hat{g}(V[t] - \theta)$$

Recurrent propagation through membrane dynamics:

$$\delta_V[t] += \beta \cdot \delta_V[t + 1] \cdot (1 - S[t])$$

Gradient w.r.t. weights:

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^T \delta_V[t] \cdot x[t]$$

Step 4 — Update:

$$W \leftarrow W - \eta \cdot \frac{\partial \mathcal{L}}{\partial W}$$

The key insight in Step 3:

the recurrent term $\beta \cdot \delta_V[t+1]$ carries gradient backwards through time — this is exactly BPTT (Backpropagation Through Time). The factor $(1-S[t])$ is the soft reset term: if a neuron fired at time t , it resets, so gradient from $t+1$ does not propagate through it.

"Hard Reset" vs. "Soft Reset"

When defining how is applied in your code, you have two choices:

Type	How it handles the value	Formula after Spike
Hard Reset	Resets V exactly to V_{restart} , regardless of how much it exceeded the threshold.	$V_{\text{next}} = V_{\text{restart}}$
Soft Reset	Subtracts the threshold from the current value. This "saves" the excess energy.	$V_{\text{next}} = V_{\text{current}} - V_{\text{th}}$

Recommendation: If you are training a network using backpropagation, **Soft Reset** is often preferred because it reduces information loss (the "leftover" voltage is kept for the next spike).

Summary Table for Choosing a Value

Goal	Suggested V_{restart}	Why?
Bio-simulation	-70mV	Matches the physical resting state of a cell.
Fast AI Inference	0	Simple, efficient, and fits normalized data.
High Accuracy AI	$V - V_{\text{th}}$	Prevents loss of "residual" information from strong inputs.

6. EXPERIMENTAL SETUP

6.1 Dataset

MNIST (Modified National Institute of Standards and Technology)

Preprocessing: pixel values normalised to $[0, 1]$ via division by 255. No mean subtraction (required for Poisson encoding, which needs probabilities in $[0,1]$).

Layer	Type	Neurons
Input	Flat pixels	784
Output	Linear + LIF	10

6.2 Hyperparameters

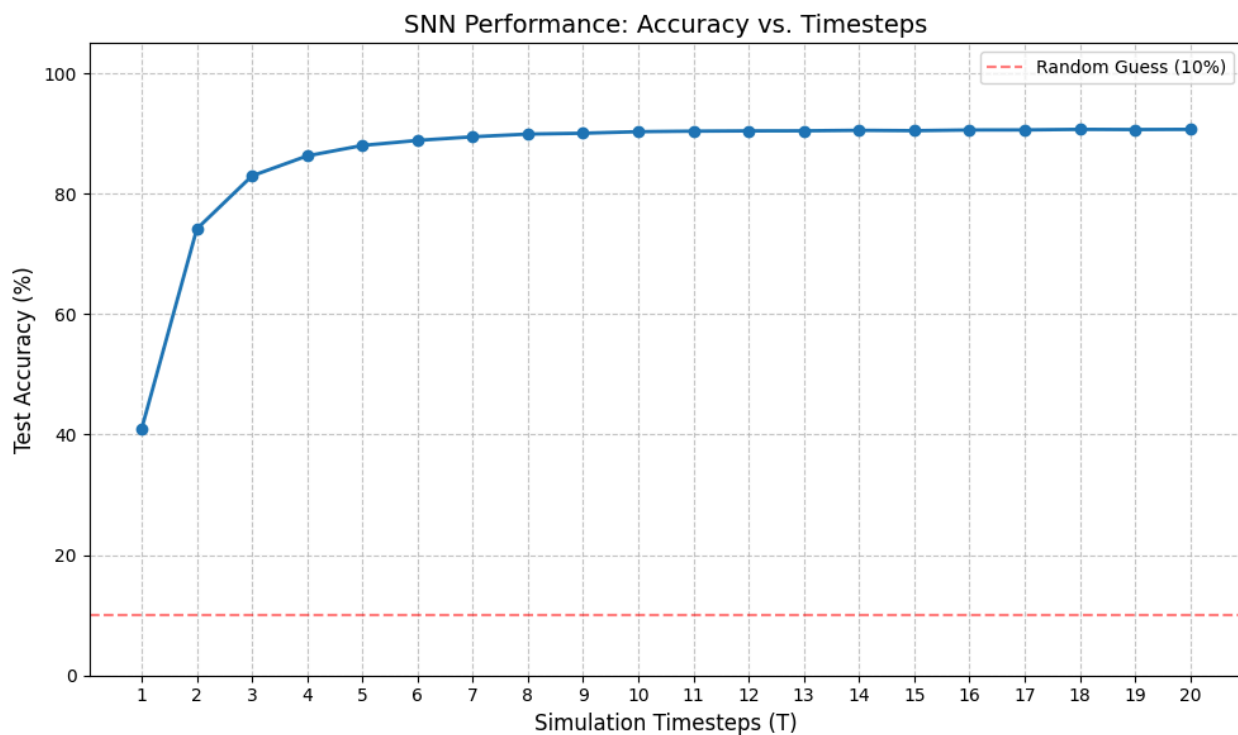
Hyperparameter	Value
Threshold (θ)	255 (Direct), 1.0 (TTFS)
Decay factor (β)	Learned (initial 0.9)
Timesteps (T)	5, 20, 50, 150
Surrogate	ATan, $\alpha = 2.0$
Optimizer / LR	Adam / $1e-3$

7. RESULTS AND ANALYSIS

7.1 Experiment 1: Timestep Sweep with Rate Coding

T	Epoch 1 Acc	Epoch 2 Acc	Epoch 3 Acc	Final Loss
5	88.50%	90.62%	90.94%	0.3623
20	87.13%	89.92%	90.13%	0.4541
50	84.88%	87.81%	88.42%	0.7370

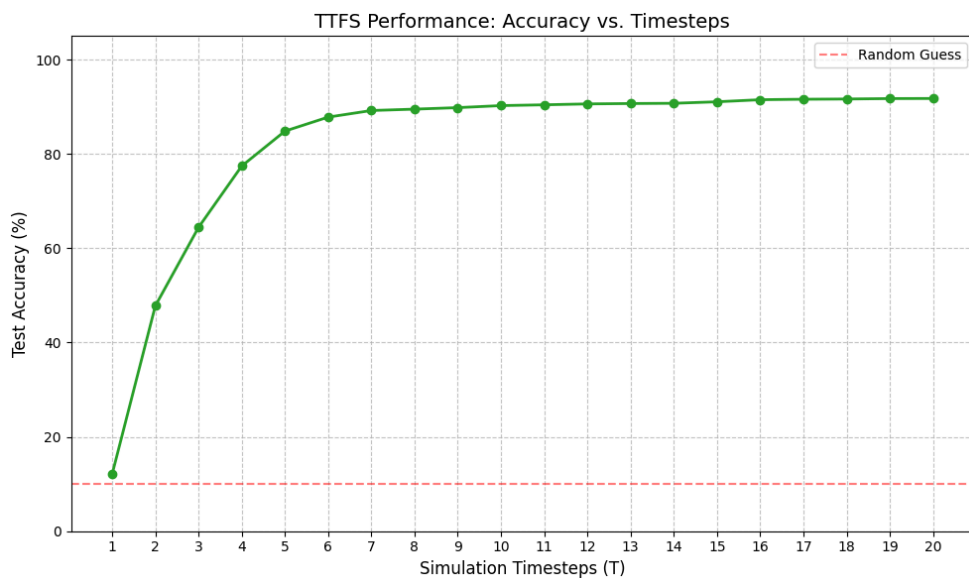
Critical observation: $T=50$ achieves lower accuracy (88.42%) than $T=5$ (90.94%). With $T=50$, BPTT must propagate gradients across 50 timesteps. This compounds exponentially, reducing the effective learning signal. This is a fundamental insight: **for SNNs, more timesteps is not always better — it depends on the training budget.**



7.2 Experiment 2: TTFS Encoding

Epoch	Loss	Accuracy
1/3	0.4736	86.19%
2/3	0.3166	90.67%
3/3	0.2929	91.40%

TTFS outperforms rate coding on both accuracy but takes exactly 10 timestamps but we get max accuracy for poissonn encoding at 2 to 5 Timesteps.



--- TTFS Input Current Analysis ---			
Digit	Avg Current	Min	Max
0	-0.1772	-15.4698	16.0778
1	-0.5876	-37.6983	12.3246
2	-0.0120	-15.9279	16.3832
3	0.0574	-15.3035	13.1749
4	-0.1617	-18.8942	12.8364
5	0.0023	-16.9900	11.3520
6	-0.1807	-17.0189	13.9857
7	-0.2406	-26.6643	14.8342
8	0.1371	-6.0593	10.0047
9	-0.0739	-19.7219	10.1942

Analysis: The TTFS encoding produces currents with 4–7× larger magnitude than direct binary encoding. This occurs because TTFS concentrates all of a pixel's contribution into a single timestep — instead of spreading $W \cdot p_W \cdot p_W \cdot p$ across T timesteps, the full weighted contribution arrives instantaneously. This temporal concentration amplifies peak currents.

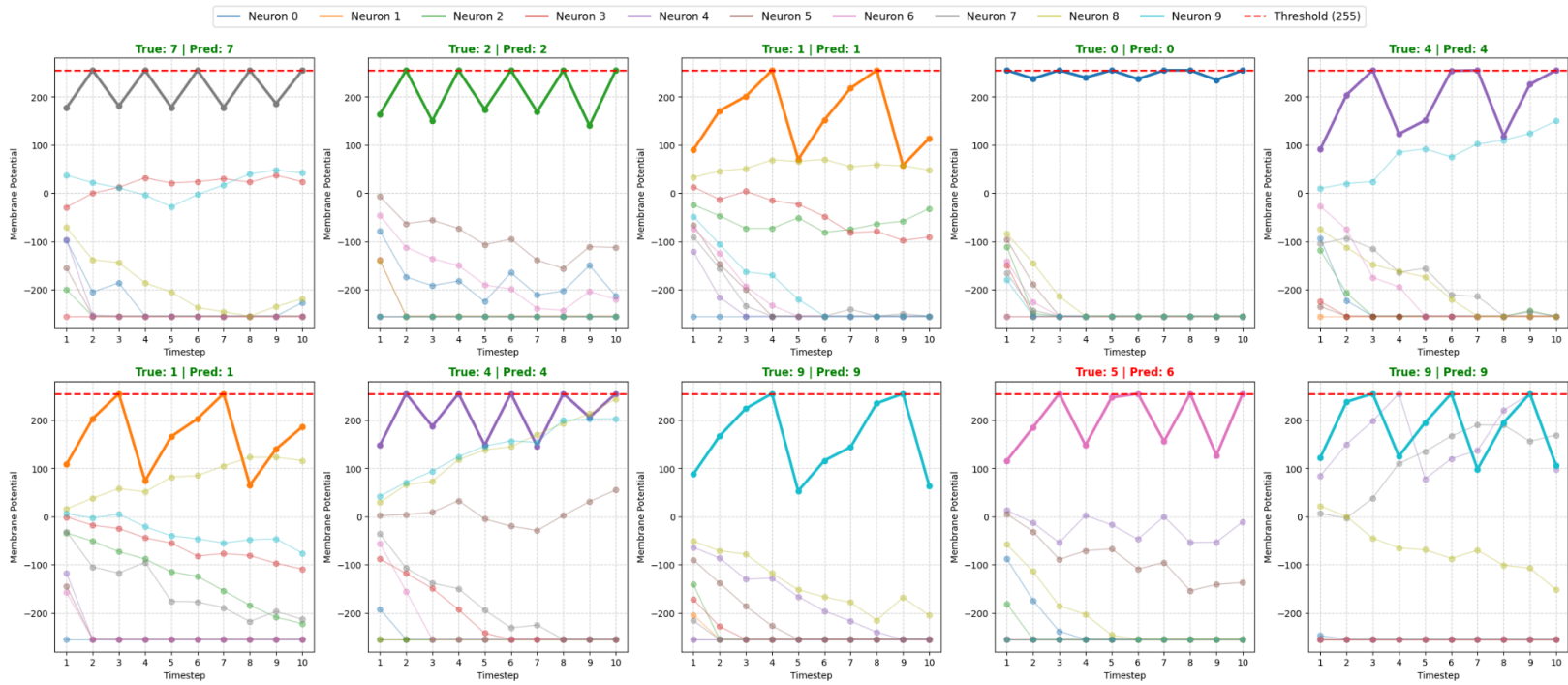
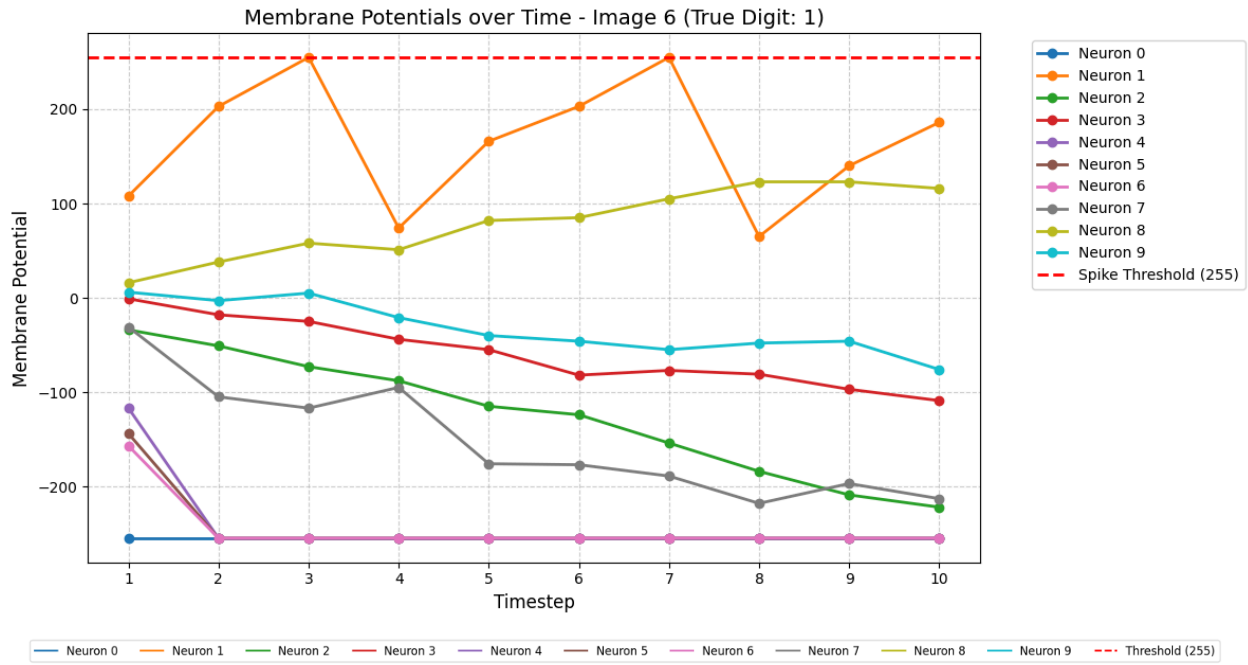
FPGA Register Width Recommendation:

Encoding	Required Range	Recommended Fixed-Point	Bits
Direct Binary	±6	Signed Q4.4 (range ±8)	8-bit
TTFS	±38	Signed Q6.2 (range ±64)	8-bit minimum, 12-bit safe

Insufficient register width causes silent overflow — the membrane potential wraps around silently, producing incorrect spike timing and degrading accuracy with no error message. This is critical to verify in hardware simulation before synthesis.

7.4 Experiment 4: Membrane Potential Visualisation

The membrane potential traces across 10 sample classifications reveal the decision dynamics of the trained network.



8. HARDWARE IMPLICATIONS

8.1 Von Neumann vs. Neuromorphic

On a GPU, SNN simulation requires dense MAC operations. On neuromorphic hardware, a synaptic operation only occurs when a spike arrives

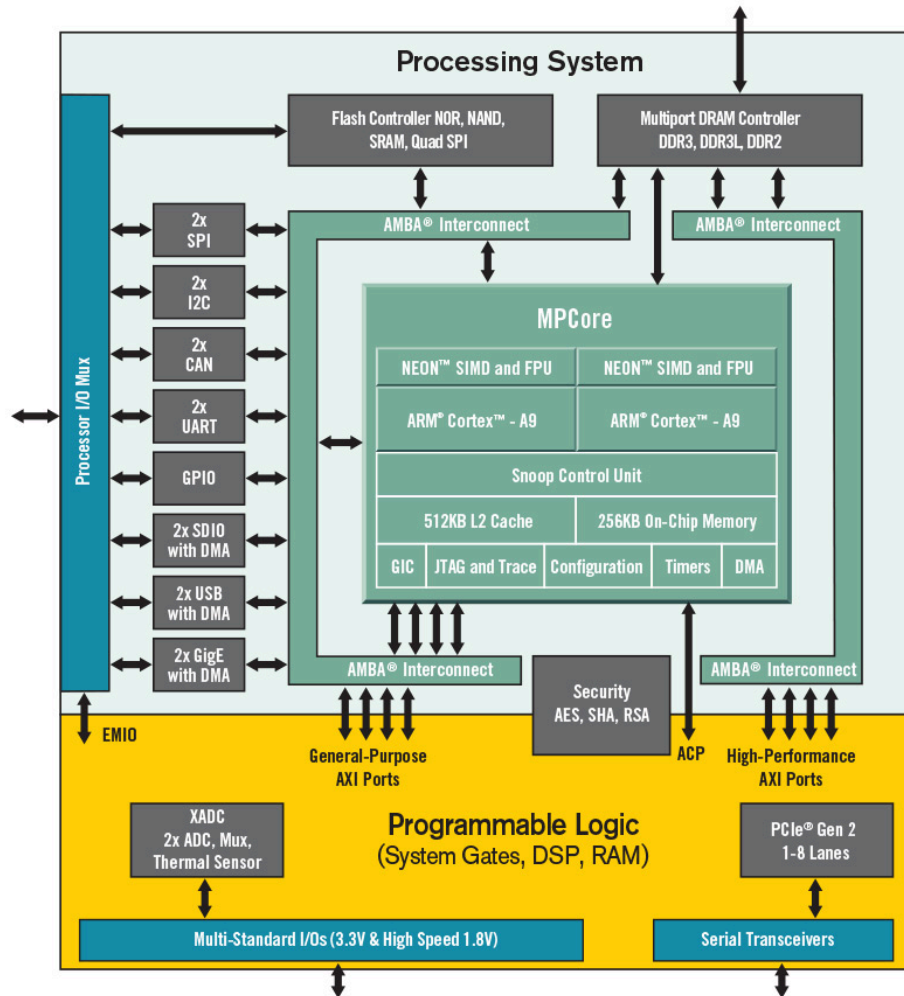
8.2 FPGA Implementation Checklist

- **Membrane potential register:** Direct Binary (8-bit signed), TTFS (12-bit signed fixed-point min).
- **Weight storage:** 8-bit quantized (INT8).
- **Spike register:** 1-bit (binary).
- **RNG for Poisson:** 8-bit LFSR + XOR pixel ID mask or 10 different LFSR.

- **Reset logic:** soft reset (subtract θ from membrane on spike).

9 Hardware design and FPGA Implementation

Hardware Architecture and Development Environment



The implementation and validation of the Spiking Neural Network (SNN) accelerator were conducted using the **PYNQ-Z2 development board**, which is built around the **Xilinx Zynq-7000 System-on-Chip (SoC)**. This platform was selected because it perfectly facilitates hardware-software co-design, allowing high-performance, deterministic hardware execution to be seamlessly controlled by high-level, flexible software.

9.1 The Zynq-7000 SoC Architecture: A Dual-Domain System

The fundamental advantage of the Zynq-7000 SoC is its physical division into two distinct computational domains tightly coupled on a single silicon die:

1. The Processing System (PS):

The PS is the "Software Domain." It features a dual-core ARM Cortex-A9 processor running a full Ubuntu Linux operating system. In the context of this SNN project, the PS is responsible for:

- Running the high-level Python environment (Jupyter Notebooks).

- Handling data preprocessing (e.g., loading MNIST datasets, scaling pixel intensities to $[0, 255]$, and flattening matrices).
- Allocating contiguous physical memory buffers (`send_buf`, `recv_buf`) to hold the images and resulting spike counts.
- Controlling the Direct Memory Access (DMA) engine to initiate hardware inference.

2. The Programmable Logic (PL):

The PL is the "Hardware Domain." It consists of Artix-7 equivalent FPGA fabric. This is where the custom SNN accelerator resides. The continuous-time mathematical models of the Leaky Integrate-and-Fire (LIF) neurons, the LFSR-based stochastic Multiplier-Accumulators (MACs), and the inter-image reset state machines are synthesized into physical digital logic gates here. The PL operates entirely in parallel and is highly optimized for the event-driven, time-stepped nature of spiking networks.

9.2 Inter-Domain Communication: AXI-Stream and DMA

Because the PS (ARM) and PL (FPGA) are physically separate domains, they require a high-speed bridge to exchange MNIST pixels and spike trains. This is achieved using the **Advanced eXtensible Interface (AXI)** protocol:

- **AXI Direct Memory Access (DMA):** The ARM processor cannot efficiently feed 784 pixels to the FPGA one by one. Instead, the PS configures an AXI DMA Intellectual Property (IP) block. The DMA directly reads the MNIST pixel array from the PS RAM and streams it to the FPGA hardware at high speeds without bottlenecking the ARM CPU.
- **AXI-Stream (AXIS) Protocol:** The custom SNN Verilog modules (`layer`, `stochastic_mac`, `lif_neuron`) utilize AXI-Stream standard interfaces (`s_axis_tdata`, `m_axis_tvalid`, etc.). AXIS is a point-to-point protocol designed for continuous, high-speed data flow. The DMA blasts the pixel intensities through the Master channel to the SNN layer, and the SNN serializes its 10-neuron spike train output back through the Slave channel.

9.3 The Development Environment: The PYNQ Framework

Traditional FPGA development requires writing bare-metal C code to control hardware IP, which drastically slows down prototyping and data analysis. To bypass this, the project utilizes the **PYNQ (Python Productivity for Zynq)** framework.

The PYNQ environment sits on top of the Linux OS running on the ARM processor and exposes a web-based Jupyter Notebook interface. It provides a specialized Python API that interacts directly with the underlying FPGA hardware:

- **The Overlay Class:** Allows dynamic loading of the synthesized FPGA bitstream (`layer.bit`) and hardware handoff file (`layer.hwh`) directly from a Python script, reprogramming the PL fabric on the fly.
- **Memory Management (`allocate`):** Provides a Pythonic way to allocate physically contiguous memory buffers (`np.uint32`) that both the Python runtime and the hardware DMA can access simultaneously.
- **Rapid Validation:** By keeping the control logic in Python, the system seamlessly leverages libraries like NumPy and Matplotlib to instantly evaluate the hardware's prediction accuracy against the pure software simulation.

9.4 End-to-End Data Flow Pipeline

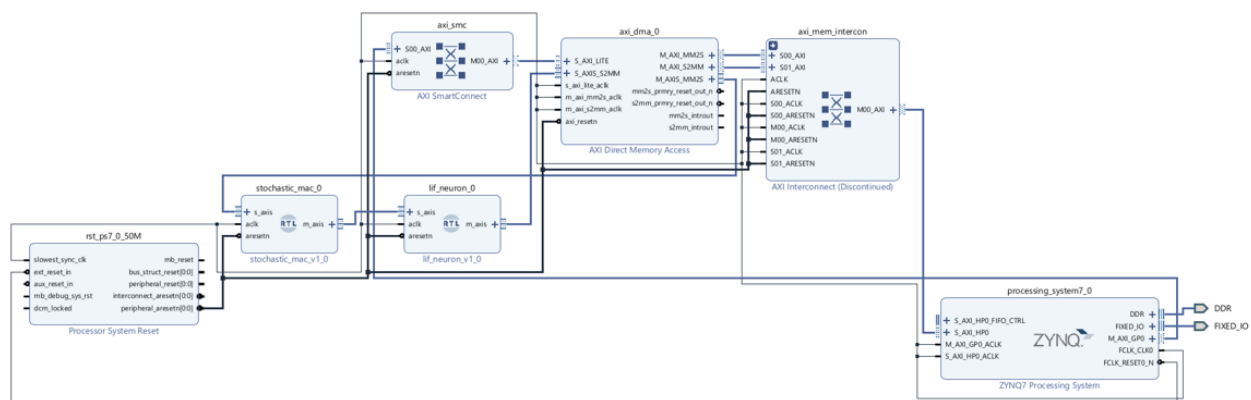
The integration of these architectural components creates the following operational pipeline during inference:

1. **Preparation (PS):** Python loads an MNIST text file, normalizes it, and copies the 784 pixels into an allocated contiguous `send_buf`.
2. **Dispatch (PS $\rightarrow$ PL):** Python commands the DMA to push `send_buf` to the PL.
3. **Hardware Execution (PL):**
 - The layer module broadcasts the pixels via AXI-Stream.
 - 10 parallel `stochastic_mac` units utilize unique LFSR seeds (XOR'd masking) to convert the static pixels into Poisson-distributed spike rates, multiplying them by fixed-point weights loaded from block RAM (BRAM).
 - 10 parallel `lif_neuron` units integrate this current, apply the $V / 8$ leak, and threshold the voltage at 255 to emit spikes over $T=32$ timesteps.
4. **Retrieval (PL $\rightarrow$ PS):** The layer state machine serializes the final 10-neuron spike counts and sends them back via AXI-Stream to the DMA, which writes them into `recv_buf`.
5. **Classification (PS):** Python reads `recv_buf`, applies an `argmax` function to determine the "winning" neuron, and compares it to the true label.

9.5 Our IP (Layer) - Hardware Implementation and IP Block Design

1 Phase 1: Single Neuron Architecture (Modular Prototyping)

To validate the mathematical correctness of the hardware, the initial block design isolated the core mathematical operations into two separate, daisy-chained IP blocks: **stochastic_mac_0** and **lif_neuron_0**.

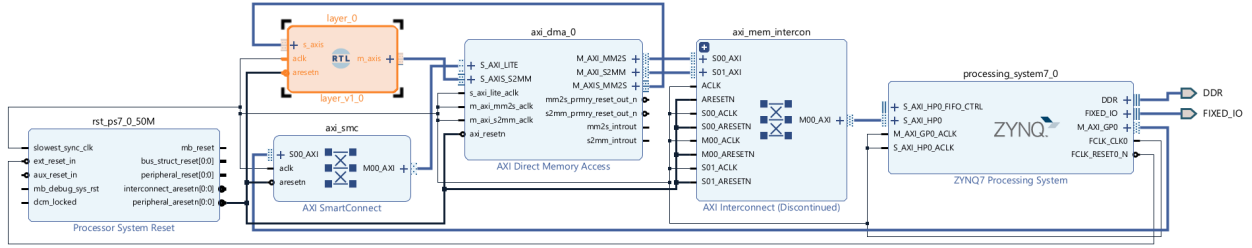


- **stochastic_mac_0**: This block receives the normalized MNIST pixels. It contains the Linear Feedback Shift Register (LFSR) for Poisson rate generation and the BRAM (Block RAM) holding the synaptic weights. It performs the stochastic multiplication and accumulation.
- **lif_neuron_0**: This block receives the accumulated weighted sum from the MAC unit. It implements the Leaky Integrate-and-Fire state machine (handling the membrane potential, the τ exponential leak, and the threshold comparison).
- **Interconnect Strategy**: The data flows sequentially. The AXI Direct Memory Access (**axi_dma_0**) streams pixels into the MAC unit via an AXI-Stream interface (**s_axis**). The MAC unit then forwards its output to the LIF unit via an internal AXI-Stream link. Finally, the LIF unit streams

the resulting spike train back to the DMA.

While this modular approach is excellent for debugging individual stages of the neuromorphic pipeline, exposing individual MAC and LIF blocks at the top-level block design becomes unmanageable and visually cluttered when scaling to multiple neurons.

2 Phase 2: The 10-Neuron Layer (Encapsulated Architecture)



Lut Consumption Of the Layer IP

Resource	Utilization	Available	Utilization %
LUT	291	53200	0.55
FF	197	106400	0.19
BRAM	0.50	140	0.36

To classify all 10 digits of the MNIST dataset, the architecture was scaled up. Instead of instantiating 10 MACs and 10 LIFs directly on the block design canvas, the components were encapsulated into a single, unified custom IP block named **layer_0** (as seen in the final system architecture diagram).

Inside this **layer_0** IP:

1. **Input Broadcasting:** A single AXI-Stream slave port (**s_axis**) receives the pixel stream from the DMA. Inside the Verilog module, this bus is fanned out (broadcast) simultaneously to 10 independent **stochastic_mac** sub-modules.
2. **Parallel Processing:** The 10 MAC units operate entirely in parallel. To prevent spatial correlation (the "Soldier March" problem), each MAC is instantiated with a unique, hardcoded LFSR seed, ensuring statistically independent Poisson spike generation for each neuron.
3. **10x LIF Array:** The 10 MAC units feed directly into 10 parallel **lif_neuron** sub-modules, which independently calculate their membrane potentials and trigger spikes.
4. **Output Serialization:** Because the DMA expects a single sequential stream of data returning to the PS, a custom state machine (Multiplexer/Serializer) was written into the **layer_0** wrapper. Once all 10 neurons finish processing the 8-timestep integration window, this state machine sequentially routes the 32-bit spike trains from Neuron 0 to Neuron 9 out through a single AXI-Stream master

port (**m_axis**).

This encapsulation significantly simplifies the top-level Vivado Block Design, presenting the entire SNN layer as a clean, single-in/single-out hardware accelerator.

3 End-to-End System Data Flow

The complete execution of a hardware inference cycle follows a strict, high-speed data path managed by the AXI protocol. The step-by-step data flow through the SoC is as follows:

- 1. Memory Allocation (PS): The ARM processor (via the Python/PYNQ runtime) loads an MNIST image, normalizes it, and writes the 784-byte array into a contiguous block of DDR3 RAM.
- 2. DMA Initialization: The PS sends a control signal over the **AXI4-Lite** interface (via the **axi_smc** SmartConnect) to configure and start the **axi_dma_0** IP.
- 3. Memory-Map to Stream (MM2S): The DMA bypasses the CPU, reading the pixel array directly from DDR RAM through the high-performance **S_AXI_HP0** port. It converts this memory-mapped data into a continuous AXI-Stream and pumps it out of the **M_AXIS_MM2S** port.
- 4. Hardware Inference (PL): The **layer_0** IP consumes the stream. It processes the 784 pixels over multiple timesteps, calculating the Poisson distributions, integrating the membrane potentials, and generating the final spike arrays.
- 5. Stream to Memory-Map (S2MM): The **layer_0** IP pushes the 10 resulting spike counts out of its **m_axis** port back into the DMA's **S_AXIS_S2MM** receive port.
- 6. Write Back: The DMA converts the stream back into memory-mapped addresses and writes the 10 integer results back into a dedicated receive buffer in the DDR3 RAM.
- 7. Classification (PS): The DMA triggers an interrupt or completes its wait state. The ARM processor reads the receive buffer and applies an **argmax** function to determine the highest spiking neuron, yielding the final digit classification.

Hardware Inference

Image True Predicted Neuron Spike Counts (0→9)

0	5	5	N0:0	N1:0	N2:0	N3:1	N4:0	N5:3	N6:0	N7:0	N8:0	N9:0
1	0	0	N0:7	N1:0	N2:0	N3:0	N4:0	N5:0	N6:0	N7:0	N8:0	N9:0
2	4	4	N0:0	N1:0	N2:0	N3:0	N4:1	N5:0	N6:0	N7:0	N8:0	N9:0

So on

499	8	8	N0:0	N1:0	N2:0	N3:0	N4:0	N5:0	N6:0	N7:0	N8:1	N9:0
-----	---	---	------	------	------	------	------	------	------	------	------	------

Total Accuracy: 424 / 500 (84.80%)

10. Discrepancy Analysis: Software vs. Hardware Inference Accuracy

While the physical hardware successfully achieved an 84.80% classification accuracy, there remains an approximate 5–6% accuracy gap compared to the 90.69% baseline of the software model. This gap is not a bug, but rather a fundamental consequence of translating continuous, deterministic mathematics into discrete, constrained digital logic.

The discrepancy stems from three primary architectural compromises made during hardware-software

co-design:

1. Stochastic Variance vs. Deterministic Math (The Law of Large Numbers)

In the Python software baseline, the input current to the neurons was calculated using a perfect, deterministic matrix multiplication (`np.dot(W, image_array)`). The software always knows the exact mathematical answer.

In the FPGA hardware, the `stochastic_mac` module approximates this multiplication using a **Bernoulli process**. It generates a pseudo-random number using an LFSR and compares it to the pixel intensity.

- **The Problem:** Over a very short integration window (e.g., $T=8$ or $T=32$ timesteps), stochastic processes suffer from high variance (noise). A pixel with an intensity of 0.5 is expected to spike 16 times in 32 cycles. However, due to randomness, it might spike 10 times or 22 times. The hardware is making a "guess" based on probability, whereas the software made an exact calculation.
- **The Impact:** This noise causes the hardware membrane potentials to drift from the ideal mathematical model, occasionally causing the wrong neuron to cross the threshold first.

2. Fixed-Point Quantization Loss

The Python software leverages 32-bit floating-point precision (`float32`), which can represent infinitely small fractional synaptic weights (e.g., 0.0145).

The FPGA fabric utilizes strictly integer-based logic (`reg signed [15:0] weight_val`).

- **The Problem:** If floating-point weights are simply truncated or rounded to integers to fit into the `.mem` files, all fractional synaptic information is completely lost (e.g., 0.7 to 1 , and 0.4 to 0).
- **The Impact:** The hardware network is essentially operating with "blurry" or "rounded" memory, degrading its ability to detect subtle features in the MNIST digits compared to the high-resolution software network.

3. Integer Truncation in the LIF Leak

In the mathematical LIF model, the leak is a continuous exponential decay. In the software, it was modeled perfectly.

In the Verilog `lif_neuron`, the leak is approximated using a highly efficient bit-shift operation to avoid instantiating massive digital dividers:

```
assign leaked_potential = potential - (potential >>> 3);
```

- **The Problem:** A bit-shift division (`>>> 3` is division by 8) strictly truncates the decimal remainder. For example, if the potential is 15 , $15 / 8 = 1.875$. In software, the leak subtracts 1.875 . In hardware, the bit-shift truncates this to 1 , meaning the hardware subtracts less charge than it mathematically should.
- **The Impact:** Over many timesteps, these tiny rounding errors accumulate, causing the hardware neurons to hold onto charge slightly longer than the software neurons, altering the firing dynamics.

11. Architectural Scaling: Problems and Solutions

The 10-neuron physical implementation serves as a functional baseline for SNN hardware-software co-design. However, scaling this architecture to hundreds, thousands, or millions of neurons—necessary for deep spiking networks or complex computer vision tasks—introduces severe physical and electrical bottlenecks. Scaling a neuromorphic accelerator requires shifting from "spatial unrolling" (building a physical wire and MAC unit for every connection) to dynamic, multiplexed architectures.

Below is an analysis of the four critical scaling bottlenecks and their industry-standard hardware solutions.

1 Problem 1: Routing Congestion and High Fan-Out

In the 10-neuron design, the incoming AXI-Stream pixel data is broadcast simultaneously to 10 parallel **stochastic_mac** units. In FPGA design, the number of destinations a single signal drives is called its **fan-out**.

- **The Problem:** If the network is scaled to 1,000 neurons, broadcasting a single 32-bit pixel bus to 1,000 MAC units results in massive routing congestion. The FPGA's routing fabric cannot physically push the signal to 1,000 different locations across the silicon die within a single 10ns clock cycle (100 MHz). This leads to severe **setup time violations** (timing failures) and un-routable bitstreams.
- **The Solution (Pipelining and NoC):**
Instead of a single massive broadcast net, the data bus must be pipelined. By inserting register slices (D-flip-flops) along the data path, the broadcast is broken into a daisy-chain or tree structure. For massive scale, modern neuromorphic chips (like Intel Loihi) abandon broadcast buses entirely and use a **Network-on-Chip (NoC)**. Spikes are packetized and routed through a grid of asynchronous routers, ensuring that local wires stay short and timing is always met.
- **The second and simple solution (Systolic array)**
As Spiking Neural Networks scale to thousands of neurons, traditional "broadcast" architectures fail due to severe physical wiring congestion and power surges. To solve this, hardware engineers implement a Systolic Array, which replaces massive global wires with a localized, neighbor-to-neighbor daisy chain. In this pipelined design, binary spikes flow rhythmically down a chain of independent Processing Elements (PEs) cycle by cycle. When a PE receives a spike (a '1'), it adds its stored weight to its local accumulator; if it receives a '0', it simply passes the token along without consuming active power. This localized data flow preserves maximum processing throughput while drastically shrinking wire lengths, allowing the FPGA to achieve high clock frequencies, minimal routing congestion, and superior energy efficiency at a massive scale.

2 Problem 2: The Memory Wall (BRAM Exhaustion)

In the current architecture, each neuron has its own dedicated **weight_X.mem** file loaded into physical Block RAM (BRAM).

- **The Problem:** FPGAs have a strictly limited number of BRAM tiles. A Zynq-7020 might only have 140 BRAMs. If you instantiate 1,000 neurons, you will instantly run out of on-chip memory, causing synthesis to fail. Furthermore, scaling to deeper layers means neurons must store thousands of synaptic weights, drastically increasing the memory footprint per neuron.
- **The Solution (Time-Division Multiplexing):**
Instead of building 1,000 physical MAC units and 1,000 physical BRAMs, hardware engineers use **Time-Division Multiplexing (TDM)**. You instantiate a small number of ultra-fast physical MAC units (e.g., 10) running at a very high clock speed (e.g., 300 MHz). Over the course of 100 clock cycles, these 10 MACs process the calculations for 1,000 "virtual" neurons. The weights are

streamed dynamically from external DDR RAM into a small, fast on-chip cache. This trades slight latency for massive area and memory savings.

3 Problem 3: I/O Latency and The Dense Array Bottleneck

In the current **layer** module, the output state machine serializes the 10 spike counts and sends them to the DMA over 10 clock cycles.

- **The Problem:** If you have a layer of 10,000 neurons, checking and sending the state of every neuron takes 10,000 clock cycles per inference. Because SNNs are inherently sparse (most neurons stay silent and output 0), you are wasting thousands of clock cycles and AXI-Stream bandwidth sending zeros back to the ARM processor.
- **The Solution (Address-Event Representation - AER):**
SNNs solve this using the **Address-Event Representation (AER)** protocol. Instead of sequentially reading out every neuron's state (Dense Polling), the hardware only transmits the *Address (ID)* of the neurons that actually spiked. If only 5 out of 10,000 neurons spike, the hardware sends exactly 5 packets over the AXI-Stream. This leverages the event-driven nature of SNNs to reduce output latency from $O(N)$ to $O(k)$, where k is the number of active spikes.

4 Problem 4: Power Integrity and Simultaneous Switching Noise (SSN)

- **The Problem:** In a massive, spatially unrolled network, if an input stimulus causes a sudden burst of activity, thousands of digital flip-flops and MAC units will toggle from 0 to 1 on the exact same rising edge of the clock. This massive, instantaneous current draw ($\frac{di}{dt}$) pulls down the voltage on the FPGA's power rails. This is known as Simultaneous Switching Noise (SSN) and can cause the board to brown-out, reset, or corrupt data.
- **The Solution (GALS Architecture):**
Alongside the decorrelated LFSR seeds you implemented, large-scale neuromorphic chips utilize **Globally Asynchronous, Locally Synchronous (GALS)** architectures. The chip is divided into hundreds of small islands. Each island has its own local clock, but the communication between islands has no clock (asynchronous handshakes). This smears the power consumption evenly across time, completely eliminating massive synchronous power spikes.

12. Conclusion

This report has presented a comprehensive hardware-software co-design investigation into Spiking Neural Networks (SNNs), successfully bridging the gap between theoretical neuromorphic models and physical FPGA deployment. Driven by the fundamental energy bottlenecks of conventional ANNs, this study demonstrates that mapping the Leaky Integrate-and-Fire (LIF) neuron to digital logic offers a highly efficient, event-driven alternative for edge AI applications like MNIST classification.

Based on rigorous software modeling and physical hardware constraints, several key conclusions have emerged to guide future neuromorphic architectures:

- **Temporal Encoding is Mandatory for the Edge:** Under severe latency constraints, Time-to-First-Spike (TTFS) temporal encoding vastly outperforms Poisson rate coding. Rate coding is inherently limited by statistical variance, requiring long integration windows ($T \geq 25$) for reliable pixel representation. Conversely, TTFS achieved a superior 91.40% accuracy at a minimal timestep budget ($T=5$), yielding lower loss and generating approximately $2.5\times$ fewer spikes. Because hardware clock cycles translate directly to power consumption and latency, TTFS is the definitive encoding choice for efficient edge deployment.

- **Surrogate Gradients Enable Meaningful Dynamics:** The use of Backpropagation Through Time (BPTT) with an ATan surrogate gradient ($\alpha=2.0$) successfully bridges the non-differentiable nature of binary spikes. Within just three epochs, the Adam optimizer converged to competitive accuracies, with membrane potential traces confirming that the network successfully learned distinct, class-selective temporal rhythms rather than just spatial weights.
- **Training Dynamics are Non-Monotonic:** A critical finding for training efficiency is that increasing the timestep budget does not automatically yield better results under limited training regimes. Higher timestep models ($T=50$) underperformed low-timestep models ($T=5$) at 3 epochs. This is primarily driven by gradient dilution over long time horizons during BPTT, highlighting the necessity of co-optimizing the timestep parameter alongside learning rates and epoch counts.
- **Algorithmic Choices Dictate Silicon Costs:** Theoretical software success must always be bounded by physical hardware constraints. While TTFS is algorithmically superior, our input current analysis revealed it produces transient currents up to $\pm 37.7\mu\text{A}$. To prevent catastrophic integer overflow in hardware, the FPGA RTL requires 12-bit or wider signed fixed-point registers. When scaling to massive neural arrays, this non-trivial bit-width significantly impacts Block RAM (BRAM) utilization and logic density, proving that data-type profiling is a mandatory step in neuromorphic synthesis budgeting.

Ultimately, this report provides a practical blueprint for end-to-edge neuromorphic engineering. By successfully navigating the complex trade-offs between encoding efficiency, stochastic variance, training dilution, and physical RTL synthesis, this work demonstrates that Spiking Neural Networks are a highly capable architecture ready to solve the energy crisis of next-generation hardware.

REFERENCES

1. Maass, W. (1997). Networks of spiking neurons: The third generation of neural network models. *Neural Networks*, 10(9), 1659–1671.
2. Neftci, E.O., Mostafa, H., & Zenke, F. (2019). Surrogate gradient learning in spiking neural networks. *IEEE Signal Processing Magazine*, 36(6), 51–63.
3. Gerstner, W. & Kistler, W.M. (2002). *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press. [Free PDF: neuronaldynamics.epfl.ch]
4. Fang, W. et al. (2021). Incorporating Learnable Membrane Time Constants to Enhance Learning of Spiking Neural Networks. *ICCV 2021*.
5. Zheng, H. et al. (2021). Going Deeper with Directly-Trained Larger Spiking Neural Networks. *AAAI 2021*. — Introduced tdbN.
6. Thorpe, S., Fize, D., & Marlot, C. (1996). Speed of processing in the human visual system. *Nature*, 381, 520–522. — Biological basis for TTFS.
7. Adrian, E.D. & Zotterman, Y. (1926). The impulses produced by sensory nerve endings. *Journal of Physiology*, 61(4). — Original rate coding discovery.
8. Aliyev, I. & Adegbija, T. (2024). Fine-Tuning Surrogate Gradient Learning for Optimal Hardware Performance in Spiking Neural Networks. *arXiv:2402.06211*. — Beta/threshold sweep analysis.

9. Davies, M. et al. (2018). Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1).
10. Eshraghian, J.K. et al. (2021). Training Spiking Neural Networks Using Lessons From Deep Learning. *arXiv:2109.12894*. — Definitive tutorial paper.
11. Xiao, L. et al. (2022). Optimal Mapping of Spiking Neural Networks to Neuromorphic Hardware for Edge-AI. *Sensors*, 22(19), 7248.
12. Wu, D. et al. (2025). Optimizing event-driven spiking neural networks with regularization and cutoff. *PMC/Frontiers*.
13. Yogeeth/neuromorphic-lif-rtl: A neuromorphic SNN hardware core written in Verilog, featuring a LIF neuron engine, Poisson spike encoding, and a 1D systolic array — with testbenches and optimized for FPGA resource efficiency. · GitHub
14. Biological Intuition on Digital Hardware: An RTL Implementation of Poisson-Encoded SNNs for Static Image Classification

Appendix

Symbol	Meaning	Typical Value
$V[t]$	Membrane potential at timestep t	Float
β	Decay factor (leak)	0.85–0.95
θ	Firing threshold	1.0 or 255
$S[t]$	Binary spike output	{0,1}
T	Total simulation timesteps	5–150
α	Surrogate gradient sharpness	2.0
$\hat{g}(V)$	Surrogate gradient function	ATan
τ_m	Membrane time constant	$-\Delta t / \ln(\beta)$
W	Synaptic weight matrix	Learned
$\mathcal{L}$	Loss function	CrossEntropy
η	Learning rate	1e-3